Sports Edge

A fully autonomous, multi-agent AI betting desk — technical overview for developers

Internal architecture briefing · prepared for a technical / engineering reader · July 2026

Sports Edge is a sports-betting *information* business that is operated almost end-to-end by a team of specialised AI agents. What makes it interesting from an engineering standpoint is not the betting — it's that the entire back office (research, publishing, settlement, auditing, supervision, customer ops) is implemented as a set of **scheduled Claude agents that coordinate through a shared document store**, with strict separation of duties, no central application server, and a human kept deliberately in the loop for the one irreversible action: placing money.

1 · What the product actually is

The product is not "winning picks." It is a **verifiable, honest track record**. The whole thesis is that the tipster market is full of unverifiable records, deleted losses, and cherry-picked screenshots — so Sports Edge competes on *trust*: every selection is published before the event starts with a stated minimum acceptable price, every result (including losses and voids) is recorded and never edited, and the desk grades itself publicly on closing-line value (whether the market later agreed its price was sharp) rather than on win rate. "No bet today" is treated as a valid, publishable product.

> **Current maturity (stated honestly):** a handful of days of live record, ~18 settled bets, small positive P/L. The desk's own auditor calls this *variance, not proven edge*. Everything in the design — the gates, the caps, the never-delete rules — follows from taking that uncertainty seriously.

2 · The core architectural idea

There is no traditional backend. Each "employee" is an **independent AI agent** that wakes on a schedule (a cron-style trigger), does a narrow job, writes its results into shared files, and exits. The design borrows from how a real, well-governed company separates responsibilities:

- **Separation of duties.** No single agent selects a bet, publishes it, settles it, *and* reports on it. The analyst cannot settle its own bets; the settlement agent cannot pick or re-price; the auditor cannot be edited by the desk it audits.
- **Shared state instead of an API.** Coordination happens through a versioned knowledge base of markdown/CSV/HTML documents (the "project"). Every writer stamps a timestamp and its sources; the last writer reconciles running totals. This is effectively an append-and-reconcile ledger, not a mutable database.
- **Management by exception.** Supervisory agents stay *silent* when everything is correct and only emit a notification when a rule is violated — so the human sees signal, not noise.
- **Human-in-the-loop for irreversible actions.** The agents research, price, publish and settle, but a person places every wager by hand and reports back the actual odds taken. No agent ever moves money.

3 · The agent roster and schedule

Agents split into a betting/research department and a business-operations department. Each scheduled agent is a Claude task bound to a cron trigger; the betting desk runs the following live cadence:

Agent	Role	Schedule (cron / cadence)	Writes
Clive	Lead analyst — the only agent allowed to select bets	9 PM evening card, 2 AM scout, 4:30 AM day-card + settle	Cards, reasoning, experiments
Otto	Back-office ops — settles results, keeps records true	`46 * * * *` (hourly)	Ledger, CSV, dashboard
Otto-B	Live-window checks; flags finished/unsettled events	`16 * * * *` (hourly)	Provisional flags for Otto
Vera	Daily supervision — verifies the work was done right	10 AM + 4 PM sweeps	Exception findings only
Vera-W	Lightweight hourly watchdog (heartbeats, stale data)	`31 * * * *` (hourly)	Alerts only; silent when green
Ada	Independent weekly auditor — reconciles everything	`0 0 * * 1` (Sun 8 PM ET)	Weekly honesty report
Business side (design-complete, staged rollout): Maya (chief of staff), Grace (compliance / safer-gambling), Sofia (content), Theo (Telegram & subscriptions), Holly (support), Finn (finance).			

> **Why the redundancy (Otto at :46, Otto-B at :16, Vera-W at :31):** settlement and freshness are the desk's core reliability guarantees. Overlapping cheap checks give a ~30-minute effective cadence so no finished bet stays unsettled for long, with an independent watchdog on top.

4 · Data model / shared state

The "system of record" is a set of files, not a database. The important ones:

- `ledger.md` — the human-readable source of truth: every card, every settled result with two sources, running units/dollars/bankroll, an ops log of changed-runs, and a source-reliability flag archive.
- `bets.csv` — a structured mirror of every bet (id, odds, stake, est/no-vig probability, close, outcome, P/L, status) for programmatic reconciliation and calibration.
- `live-dashboard-template.html` — a self-contained dashboard that fetches live scores from a public feed *in the reader's browser*; the agents re-send it after material changes.
- Rule/knowledge docs — staking rules, pro-betting principles, the org charter, source playbooks, calibration code. These act as durable long-term memory that every future agent run reads before acting.

Reconciliation is disciplined rather than transactional: writers stamp time and source, changed-only logging avoids churn, and the auditor independently re-derives totals weekly. Divergence between `ledger.md` and `bets.csv` is a *finding*, never silently patched.

5 · Key technical workflows

5.1 Pick generation

For each slate the analyst fans out **parallel research sub-agents** (one per sport/tournament group), each gathering matchups, current odds, model/expert probabilities, form, injuries and fatigue. Results are synthesised into a probability estimate per candidate, compared against the no-vig implied probability to compute edge/EV, and filtered through confidence and data-sufficiency gates. Outputs carry an advised price, a **minimum acceptable price** (below which the human skips → the bet voids rather than counts), stake in units, evidence for and against, and a confidence grade.

5.2 Settlement (the hardest engineering problem here)

Confirming a final score reliably is genuinely non-trivial: sports aggregators serve stale caches, "decoy" recaps of prior meetings rank highly in search, and some feeds show false placeholders. The desk enforces a **two-source, dateline-verified rule** — a result is only booked when two independent sources agree on it *and* the article dateline / opponent match the actual fixture. A large "source-reliability flag" archive encodes which providers can be trusted for which task and which are cache offenders. Target: no finished bet unsettled beyond 45 minutes during a live window. Settlement writes to ledger + CSV + dashboard atomically in one run.

5.3 Self-grading: CLV and calibration

The desk captures a near-close price for every bet to compute **closing-line value** (did the market move toward or against the pick?), and tracks a **Brier score** of its probability estimates against the market's implied probabilities. These are the real performance measures — win rate alone is explicitly rejected, because a lucky win on bad reasoning is logged as a poor decision and a well-reasoned loss is not.

5.4 Supervision

Watchdog agents run management-by-exception: they check pending-bet freshness, deadline trackers, agent heartbeats and dashboard age, escalate by severity (minor → operational → material → critical),

avoid repeating the same alert, and stay completely silent when everything is green.

6 · Governance & safety model

- **Never-edit records.** Losses and voids are permanent; history is append-only with audit entries. One hidden loss would destroy the product's entire premise.
- **No end-to-end control.** Selection, publication, settlement and reporting are owned by different agents by design.
- **No auto-placement.** Agents never place bets or move funds; a human executes and reports actual fills. This keeps the only irreversible action under human control.
- **Responsible gambling is built in, not bolted on:** unit staking with hard caps, no loss-chasing / no stake-scaling to recover, a banned-claims list (no "guaranteed", "risk-free", "cannot lose"), and a distress protocol that signposts support instead of picks. Monetisation is gated behind a hard bar (90+ days of unbroken record AND n ≥ 200 settled bets AND positive CLV AND non-negative P/L) — until then the full product is free.

7 · Tech stack, in one view

Layer	Implementation
Compute / reasoning	Claude (Opus-class) agents, one per scheduled role
Scheduling	Cron-style triggers (managed scheduled-task service); each firing is a fresh agent session
Coordination / "database"	A shared, version-stamped project knowledge base (markdown + CSV + HTML) — no bespoke server
Research I/O	Web search + page fetch tools; parallel sub-agents for fan-out
Presentation	Self-contained HTML dashboard with client-side live-score fetch
Human interface	Push/email notifications on exceptions; the human places bets and reports fills

8 · Engineering challenges worth knowing about

- **Source trust & cache defeat.** Much of the settlement logic is really an information-verification problem — beating stale caches and decoy content with dateline + opponent + two-source checks.
- **Idempotent, atomic multi-file writes.** Ledger, CSV and dashboard must stay consistent even though each agent run is stateless and may overlap another.
- **Race conditions between overlapping runs.** "Last writer reconciles" plus changed-only logging is the current mitigation; the weekly audit is the backstop.
- **Scheduling cost vs coverage.** Three hourly jobs alone are ~72 agent runs/day. That density is what drives reliability — but it also consumes the account's usage budget, and when that budget is exhausted the scheduled runs fail silently. Trimming overnight hours roughly halves the load with little coverage loss.

> **Known live constraint:** because every firing is a full agent session, a busy schedule can hit the account's usage/credit ceiling, after which scheduled runs stop producing until the limit resets or the cadence is trimmed. This is the main operational limiter of the current design, and the clearest lever for a developer to tune.

Sports Edge — internal technical overview. Describes an autonomous multi-agent system built on scheduled Claude agents and a shared document store. 18+; the underlying activity is gambling, which carries risk of loss — the product's design goal is honesty and transparency, not guaranteed returns.